

Projet final — Back-end de WebApp adossé à MongoDB

Module MIA4 · Conception et intégration d'un SGBD NoSQL — IPSSI Montpellier Jour 5 / 5 —
Cadrage et réalisation encadrée (1 journée)

L'énoncé officiel

« **Mettre en place un back-end de WebApp adossé à une base NoSQL.** » Le projet est réalisé **en binôme** et consiste à : créer, administrer et indexer une base MongoDB portant des **données réelles** ; développer un **back-end (API)** exposant les opérations CRUD ; produire des **agrégations** répondant à des questions métier ; un **front-end minimal** affiche les résultats (tableau, carte ou graphique). *Extrait de la fiche de cours IPSSI — Mastère Dév, Data & IA, 4^e année.*

Vous avez passé quatre jours à apprendre les briques : CRUD et modélisation document, index et `explain()`, réplication et sharding, pipelines d'agrégation, géospatial, diagnostic et sécurité. Aujourd'hui, vous les assemblez sur **un cas réel de votre choix**.

L'évaluation passe par un passage de validation en direct, en visio. Vous avez la journée entière pour construire ; en fin d'après-midi, chaque binôme rejoint le formateur en visio, **partage son écran**, fait tourner son projet devant lui et répond à ses questions. **Rien n'est évalué sur un rendu que vous n'auriez pas su faire fonctionner devant lui.**

Ce qui est évalué n'est pas la beauté du front-end. C'est votre capacité à **justifier chaque décision technique par une mesure ou par un besoin métier**.

1. Cahier des charges — le socle à couvrir

Les six points suivants constituent le **minimum évaluable**. Tout le reste est bonus.

#	Exigence	Ce qui est vérifié au passage de validation
1	Jeu de données réel, ≥ 10 000 documents	Vous montrez la source publique et la commande d'import. Des données générées par Faker ou saisies à la main disqualifient le projet.
2	Au moins 2 collections liées	Vous expliquez, schéma à l'appui, pourquoi vous avez imbriqué ici et référencé là. Au moins un <code>\$lookup</code> significatif.
3	API REST fonctionnelle	CRUD complet sur au moins une collection + routes d'agrégation. Testée en direct , sur votre machine, en partage d'écran.

#	Exigence	Ce qui est vérifié au passage de validation
4	≥ 3 agrégations métier	Chaque pipeline répond à une question formulée en français avant d'être codée. Résultats affichés côté front.
5	Index justifiés par <code>explain()</code>	Capture avant / après sur au moins une requête clé : passage <code>COLLSCAN</code> → index, avec les valeurs de <code>totalDocsExamined</code> . (Le <code>starter/</code> vous fournit de quoi produire ces deux captures — voir § 3.)
6	Sécurité de base	Authentification activée, utilisateur applicatif à privilèges limités, secrets hors du dépôt (<code>.env</code> ignoré, <code>.env.example</code> commité).

Ce qui fait la différence (facultatif)

- Déploiement **Docker Compose** complet : `docker compose up` démarre base + API + front.
- **Replica Set** local à 3 nœuds et démonstration d'un failover.
- **Index géospatial** `2dsphere` réellement exploité par le front (carte).
- **GridFS** pour des fichiers dépassant 16 Mo.
- **Tests** automatisés (pytest / jest) et **intégration continue** GitHub Actions.
- **Pagination**, validation des entrées (Pydantic / JSON Schema), gestion propre des erreurs HTTP.

2. Thématiques et jeux de données validés

Toutes les sources ci-dessous ont été **retestées le 27 août 2026** : l'URL répond et le volume annoncé a été recompté. Vous pouvez proposer un autre sujet, **sous réserve de validation par le formateur avant 10h30**.

Les jeux utilisés pendant les Jours 1 à 4 — `primer-dataset` (restaurants NYC), `sample_mflix`, `sample_training/zips`, `sample_training/trips` — sont **exclus** : le travail de découverte y est déjà fait.

A. Immobilier — « Prix au m² dans l'Hérault » (sujet local, recommandé)

Source : Demandes de Valeurs Foncières, data.gouv.fr — mutations immobilières réelles, géolocalisées.

```
curl -L -o dvf34.csv.gz https://files.data.gouv.fr/geo-dvf/latest/csv/2024/departements/34.csv.gz
gunzip dvf34.csv.gz && wc -l dvf34.csv           # 69 652 lignes (dont l'en-tête)
mongoimport --db immo --collection mutations --type csv --headerline --file dvf34.csv
```

Volume : 69 651 lignes importées — mais seulement **29 565 mutations distinctes** (`id_mutation`), une vente s'étalant sur plusieurs lots • **Champs clés** : `valeur_fonciere`, `surface_reelle_bati`, `type_local`, `nom_commune`, `code_postal`, `longitude`, `latitude`, `date_mutation`. **Questions métier possibles** : prix au m² par commune et son évolution ; top 10 des communes les plus chères ; carte des transactions dans un rayon de 5 km autour d'un point ; répartition maisons / appartements. **Difficulté à traiter** : une même mutation apparaît sur **plusieurs lignes** (une par lot ou parcelle) — il

faut regrouper par `id_mutation` avant tout calcul, sinon tous vos prix sont faux. Nombreux champs vides.

B. Santé publique — « Inspections sanitaires »

Source : `sample_training/inspections` (inspections d'établissements, New York).

```
curl -L -o inspections.json https://raw.githubusercontent.com/nee1abalan/mongodb-sample-dataset/main/sample_training/inspections.json
wc -l inspections.json # 80 047
```

Volume : 80 047 documents. **Deuxième collection** à construire par agrégation (référentiel d'établissements ou de secteurs). **Questions** : taux de non-conformité par secteur d'activité, par quartier, évolution mensuelle, établissements récidivistes.

C. Transport aérien — « Réseau de vols »

Source : `sample_training/routes` (66 985 liaisons aériennes réelles).

```
curl -L -o routes.json https://raw.githubusercontent.com/nee1abalan/mongodb-sample-dataset/main/sample_training/routes.json
```

Volume : 66 985 liaisons · relation naturelle **routes** ↔ **compagnies** ↔ **aéroports** (plusieurs collections, `$lookup` évident, graphe possible avec `$graphLookup`). **Questions** : hubs les plus connectés, compagnies par nombre de destinations, existe-t-il un vol direct de X à Y, plus court chemin en 2 escales.

D. Mobilité temps réel — « Vélib' Paris » (*le plus ambitieux*)

Source : API temps réel Open Data Paris, **1 519 stations** actualisées en continu.

```
curl -L "https://opendata.paris.fr/api/explore/v2.1/catalog/datasets/velib-disponibilite-en-temps-reel/exports/json" \
--compressed -o velib.json
```

Volume : 1 519 stations par relevé — vous atteignez les 10 000 documents en **historisant** : un script qui interroge l'API toutes les 5 minutes produit ~18 000 documents par heure. C'est un projet de **série temporelle** (bucket pattern ou collection time-series) avec `coordonnees_geo` exploitable en `2dsphere`.

Contrainte de calendrier propre à ce sujet — lisez-la avant de le choisir. Votre collecteur doit tourner **le plus tôt possible**. En pratique : faites valider le sujet dès le début du créneau de cadrage, écrivez le collecteur **avant** le reste (une vingtaine de lignes suffisent : appel HTTP, ajout d'un champ `releve_at`, `insert_many`), lancez-le au plus tard à **11h00** et laissez-le tourner toute la journée. À 11h vous avez encore 5 heures, soit **~90 000 documents** : largement de quoi. Lancé à 14h, c'est jouable mais serré ; à 15h, le sujet est perdu. C'est le seul des six sujets où **retarder le démarrage coûte le projet**. **Questions** : stations chroniquement vides ou pleines, taux de

remplissage par arrondissement, évolution horaire, carte des stations disponibles à moins de 500 m.

E. Culture — « Recommandation de films »

Source : MovieLens (GroupLens, université du Minnesota).

```
curl -L -o ml.zip https://files.grouplens.org/datasets/movielens/ml-latest-small.zip && unzip ml.zip
# 100 836 notations, 9 742 films, 610 utilisateurs – plusieurs collections liées
```

Questions : top-N par genre, films similaires par co-notation, profil d'un utilisateur, notes moyennes pondérées par le nombre de votes.

F. Naufrages et géographie — « Cartographie d'épaves »

Source : `sample_geospatial/shipwrecks` — 11 095 épaves géolocalisées.

```
curl -L -o shipwrecks.json https://raw.githubusercontent.com/neelabalan/mongodb-sample-dataset/main/sample_geospatial/shipwrecks.json
```

Projet **100 % géospatial** : `$near`, `$geoWithin` sur polygone, zones dangereuses par densité, carte Leaflet.

3. Architecture cible

Front minimal	API REST	Driver	MongoDB
HTML/JS ou React · tableau · carte · graphique	FastAPI / Express CRUD + /agg/* validation entrées	PyMongo / Mongoose pool de connexions BSON <=> JSON	Collections + index utilisateur applicatif auth activée

Un **squelette fonctionnel** vous est fourni dans le dossier `starter/` : `docker compose up` démarre MongoDB, une API FastAPI et un front statique. Il est volontairement minimal — **c'est un point de départ, pas votre projet**.

```
starter/
├─ docker-compose.yml    mongo + api + web, prêt à démarrer
├─ .env.example          les variables à copier en .env (jamais commité)
├─ db/
│   └─ 01-init-app-user.js  crée l'utilisateur applicatif (readWrite, 1 base)
├─ api/
│   ├── Dockerfile
│   ├── requirements.txt
│   └─ main.py             CRUD + 1 route d'agrégation + /health
└─ web/
    └─ index.html          appelle l'API et affiche le résultat
```

Démarrage :

```
cd starter && cp .env.example .env
docker compose up -d
curl http://localhost:8000/health # {"status":"ok", ...}
open http://localhost:3000
```

4. Livrables

Livable	Quand	Contenu attendu
1. Démonstration en direct	le jour même, en visio, en partage d'écran	<code>docker compose up</code> démarre l'ensemble en moins d'une minute. Jeu de données déjà importé. C'est le livrable principal : il conditionne tous les autres.
2. Dépôt Git public	le jour même, avant 17h00	<code>/api</code> le back-end · <code>/web</code> le front · <code>/db</code> scripts d'import et de création d'index · <code>docker-compose.yml</code> · <code>.env.example</code> · <code>README.md</code> permettant à un tiers de tout rejouer
3. Rapport PDF (8 à 12 pages)	après la séance — date limite annoncée par le formateur en début de journée, à noter ici : _____	Les cinq chapitres du syllabus : i) problématique · ii) problèmes de données rencontrés · iii) démarche · iv) résultats · v) conclusion. Un modèle est fourni : <code>MODELE_RAPPORT.md</code>

Le README doit contenir, au minimum : le sujet et les questions métier, la procédure d'installation, la description des collections et des index, la liste des routes de l'API, et la répartition du travail dans le binôme.

Le rapport se prépare pendant la journée, il ne s'écrit pas après. Vous ne retrouverez ni vos chiffres ni vos captures une semaine plus tard. Prenez vos captures `explain()` **au moment où vous créez l'index**, notez vos anomalies **au moment où vous les découvrez**, et gardez la sortie brute de vos pipelines. Un dossier `rapport/captures/` dans le dépôt suffit.

5. Déroulé de la journée

Créneau	Activité	Ce qui doit être fait à la fin du créneau
9h00 – 9h30	Rappel du Jour 4, présentation du sujet et du <code>starter/</code>	—
9h30 – 10h30	Cadrage	Sujet choisi et validé par le formateur , 3 questions métier écrites en français , dataset en cours de téléchargement
10h45 – 12h00	Modélisation & import	Schéma des collections dessiné (embed vs référence justifié), données importées, premiers <code>find()</code>

Créneau	Activité	Ce qui doit être fait à la fin du créneau
12h00 – 14h00	Pause déjeuner	—
14h00 – 15h15	Back-end & agrégations	5 à 6 routes REST qui répondent, les 3 pipelines branchés sur <code>/agg/*</code>
15h15 – 15h45	Index & optimisation	<code>explain()</code> capturé avant , index créés, <code>explain()</code> capturé après , captures rangées dans <code>rapport/captures/</code>
15h45 – 16h00	Gel du code	Démo répétée une fois en entier , README relu, dépôt poussé
16h00 – 17h00	Passages de validation	Le formateur reçoit chaque binôme en visio (~10 min). L'ordre de passage est annoncé le matin : restez joignables et continuez à documenter et à pousser en attendant votre tour

Le passage de validation, concrètement. À l'heure annoncée, **les deux membres** rejoignent la visio et celui qui héberge le projet **partage son écran entier** (pas seulement une fenêtre : le terminal, le navigateur et l'éditeur doivent être visibles). Vous faites, dans cet ordre :

1. `docker compose up -d` depuis un état arrêté, puis `curl /health` — **ça doit démarrer devant lui** ;
2. une route CRUD et une route d'agrégation appelées en direct ;
3. le front qui affiche un résultat d'agrégation ;
4. vos deux captures `explain()` , avant et après. Puis il pose ses questions (§ 6). **Les deux membres répondent**, chacun sur son périmètre.

Le distanciel ajoute deux contraintes, anticipez-les. Le partage d'écran doit être testé **avant** votre passage, micro compris — un binôme qui passe ses 10 minutes à faire fonctionner sa visio les perd. Et si votre connexion lâche en plein passage, reconnectez-vous : le formateur reprend le binôme en fin de créneau, il n'y a pas de pénalité pour une coupure réseau.

Conseil d'organisation du binôme : à 14h00, l'un consolide l'API et les pipelines, l'autre prépare le front, les captures `explain()` et la matière du rapport. À 15h45, **tout le monde arrête de coder** — un projet à 90 % qui démarre vaut infiniment mieux qu'un projet à 100 % qui plante au moment du passage.

6. Critères d'évaluation

Le formateur regarde, dans cet ordre :

Modélisation — Le choix embed / référence est-il **argumenté** par la cardinalité, la taille et les requêtes dominantes, ou subi ? Sauriez-vous dire ce qui vous ferait changer d'avis ?

Exploitation de la donnée — Les agrégations répondent-elles à de **vraies questions métier** ? Les anomalies du jeu de données ont-elles été **détectées, documentées et traitées** ? (Aucun jeu réel

n'est propre : dire « il n'y avait pas de problème » signifie que vous n'avez pas cherché.)

Performance — Chaque index est-il justifié par un `explain()` ? Savez-vous dire quelle requête il sert et ce qu'il coûte en écriture ? Un index créé « au cas où » est un défaut, pas un bonus.

Réalisation — L'API fonctionne-t-elle vraiment, avec des codes HTTP corrects et des erreurs gérées ? Le front affiche-t-il les résultats des agrégations ?

Sécurité et reproductibilité — Authentification active, rôle applicatif limité, secrets hors du dépôt, README permettant à un tiers de tout relancer.

Passage de validation — La démo démarre en direct, depuis un état arrêté. **Les deux membres** parlent et maîtrisent leur partie. Les réponses sont précises et honnêtes : « je ne l'ai pas mesuré » est une bonne réponse, l'improvisation approximative n'en est pas une.

Questions auxquelles vous devez savoir répondre

1. Pourquoi avoir imbriqué *ceci* et référencé *cela* ? Qu'est-ce qui vous ferait changer d'avis ?
2. Montrez l' `explain()` de votre requête la plus fréquente. Quel `stage` ? Combien de documents examinés pour combien retournés ?
3. Quelle anomalie avez-vous trouvée dans les données, et comment l'avez-vous traitée ?
4. Que se passe-t-il si votre collection passe de 10 000 à 10 millions de documents ? Quelle serait votre shard key ?
5. Vos identifiants de base sont-ils dans le dépôt Git ? Prouvez-le.
6. Quelle est la limite de taille d'un document BSON, et votre modèle peut-il l'atteindre ?

7. Les sept pièges qui coûtent cher

Piège	Symptôme	Antidote
Réinventer le relationnel	15 collections, des jointures partout, aucun document imbriqué	Penser document , pas ligne . Ce qui est lu ensemble est stocké ensemble
Jeu de données factice	50 documents Faker, aucune vraie question métier	≥ 10 000 documents réels , issus d'une source publique citée
Index créés au hasard	« J'ai indexé tous les champs, ça doit être rapide »	Un index = un <code>explain()</code> qui le prouve. Sinon, supprimez-le
Démo enregistrée en vidéo	« On a préféré une vidéo pour éviter les bugs »	La démo est en direct , sur votre machine, en partage d'écran. Une vidéo ne vaut aucun point
MONGO_URI commité	Le formateur lit vos identifiants dans le dépôt public	<code>.env</code> dans <code>.gitignore</code> , <code>.env.example</code> commité. Un secret publié est un secret à changer
Un seul développeur, un spectateur	Un membre ne sait pas répondre	Répartir les modules (API, front, données, rapport). Chacun présente sa partie
Le rapport repoussé à plus tard	À J+7, plus personne ne retrouve les chiffres ni les captures	Capter au moment où ça se produit , dans <code>rapport/captures/</code>

8. Pour aller plus loin (facultatif)

À traiter **seulement une fois le socle du § 1 acquis et la démo répétée**. Chacun de ces points pèse lourd au passage de validation et dans le rapport, à condition d'apporter **un chiffre mesuré**, pas une intention.

- **B1. Prouver que votre chiffre est juste.** Tout jeu réel contient un piège de comptage : dans DVF, une mutation s'étale sur plusieurs lignes (`id_mutation` répété) ; dans `inspections`, un établissement est réinspecté plusieurs fois ; dans MovieLens, un film à 3 votes ne se compare pas à un film à 3 000. Écrivez **deux** pipelines pour votre question métier principale : le naïf et le correct. Publiez les **deux valeurs** dans le rapport, avec l'écart en pourcentage, et la phrase qui explique pourquoi la première est fausse. *Un correcteur retient bien plus une équipe qui montre son erreur qu'une équipe qui n'a « rien trouvé ».*
 - **B2. Requête couverte (covered query).** Choisissez la requête la plus fréquente de votre API et construisez un index composé tel que `explain("executionStats")` renvoie `totalDocsExamined: 0` avec un stage `PROJECTION_COVERED`. Deux questions à documenter : quelle **projection exacte** avez-vous dû imposer côté API pour y arriver, et pourquoi `_id` casse-t-il la couverture s'il n'est pas explicitement exclu ?
 - **B3. Changement d'échelle.** Multipliez votre collection par 10 (réinjectez le jeu en boucle avec un suffixe, ou historisez si vous êtes sur Vélib'). Rejouez les `explain()` du § 1 point 5. Que devient `totalDocsExamined` sur la requête indexée, et sur celle qui ne l'est pas ? Le rapport **temps indexé / temps non indexé** augmente-t-il, diminue-t-il, ou reste-t-il stable ? Expliquez le résultat avec ce que vous savez de la complexité d'un parcours d'index B-tree face à un `COLLSCAN`.
 - **B4. Refuser les données invalides à l'entrée.** Ajoutez un validateur `$jsonSchema` sur votre collection principale (champs requis, types, bornes) en `validationLevel: "moderate"`, puis tentez d'insérer un document fautif via votre API. Deux points à traiter : quel code HTTP votre back-end renvoie-t-il, et surtout — **pourquoi moderate et non strict** sur une collection déjà remplie de données réelles imparfaites ? Que se passerait-il en `strict` lors d'un `update` sur un document existant non conforme ?
 - **B5. Survivre à une panne pendant la démo.** Montez votre base en Replica Set 3 nœuds (Jour 3), branchez votre API dessus avec l'URI `?replicaSet=...&retryWrites=true`, puis **tuez le primary en pleine démonstration** pendant qu'une écriture tourne. Mesurez la durée d'indisponibilité vue **par le front**, et non par `rs.status()`. Le driver rejoue-t-il l'écriture seul ? Sous quelle condition exacte `retryWrites` sauve-t-elle l'opération ?
 - **B6. Le graphe caché.** (*spécifique aux sujets C — vols — et E — films*) Utilisez `$graphLookup` pour répondre à une question qu'aucun `$lookup` ne sait traiter : « peut-on relier l'aéroport X à l'aéroport Y en au plus 2 escales, et par quels vols ? », ou « quels films sont reliés à celui-ci par une chaîne de co-notations ? ». Documentez `maxDepth`, et le temps d'exécution — c'est l'opération la plus coûteuse du cours.
-

9. Checklist avant le passage de validation

- ☐ `git clone` + `docker compose up` fonctionne **sur une machine vierge** (testez-le sur celle du binôme)
- ☐ Les 3 questions métier sont écrites **en français** dans le README, avant les pipelines
- ☐ Les captures `explain()` avant / après sont dans `rapport/captures/`, avec les valeurs de `totalDocsExamined`
- ☐ `git log -p | grep -i "mongodb://"` ne renvoie **aucun** identifiant réel
- ☐ L'utilisateur applicatif n'est pas `root` et n'a de droits que sur **sa** base
- ☐ La démo a été répétée **en entier au moins une fois**, depuis un `docker compose down`
- ☐ Le **partage d'écran et le micro** ont été testés, sur la machine qui fera tourner la démo
- ☐ **Les deux membres** sont connectés à la visio, chacun avec le dépôt ouvert de son côté
- ☐ Le jeu de données est déjà importé (ne jamais importer 70 000 documents pendant le passage)
- ☐ Les deux membres savent répondre aux six questions du § 6
- ☐ La date limite du rapport est notée (§ 4) et le dossier `rapport/captures/` est rempli